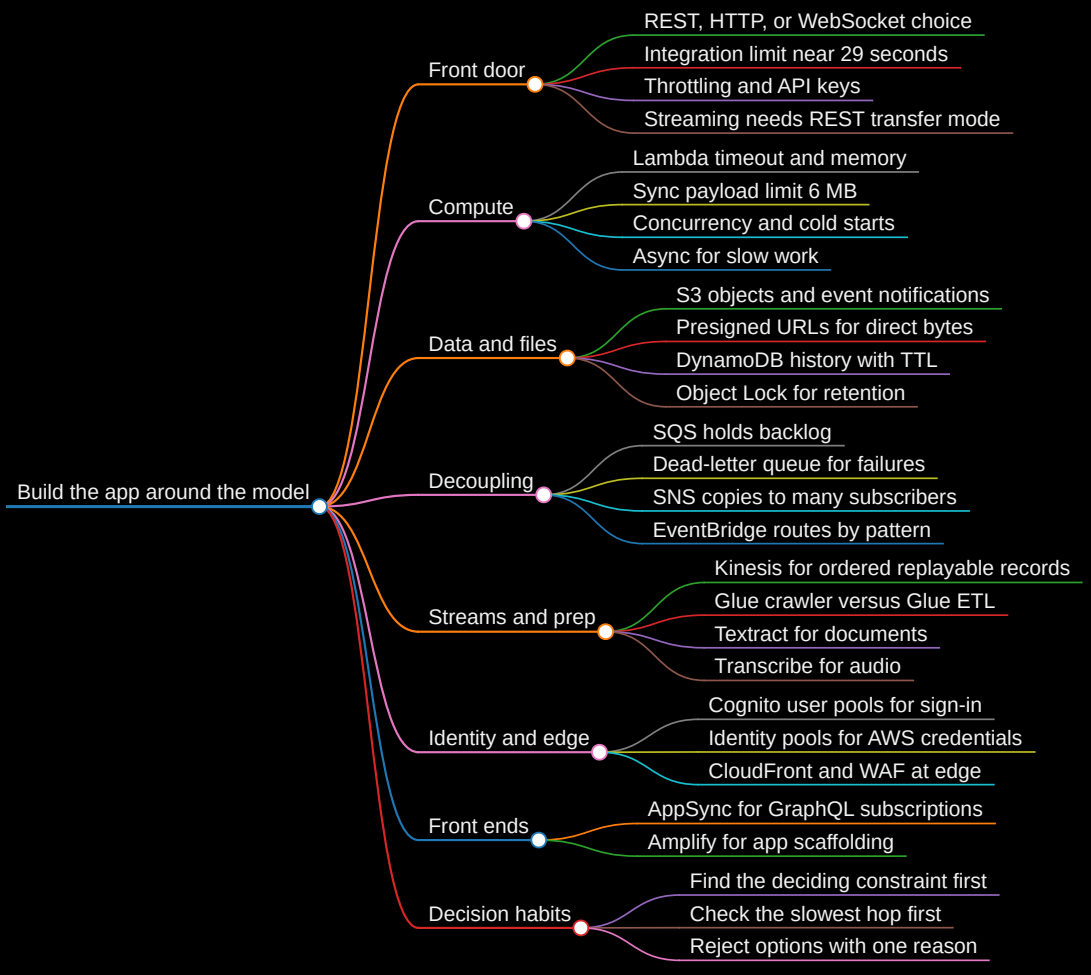
Build the App Around the Model

After this module you can read a long scenario, find the constraint that decides the service, and reject each wrong option with one reason. The deciding constraint is usually the caller wait time, the payload size, the consumer count, or the need to replay. Most wrong options are real services that solve a different hop in the request path.



The deciding constraint

Start from the scenario, not from the service names. Ask what the caller waits for, how large the data is, and how many consumers need the event. The first answer usually removes most of the options.

The order of the checks matters. A caller that must wait rules out a batch design before storage details matter. A payload that is too large rules out a direct API body before queue details matter. Check the hardest limit first, then choose the service that owns that limit.

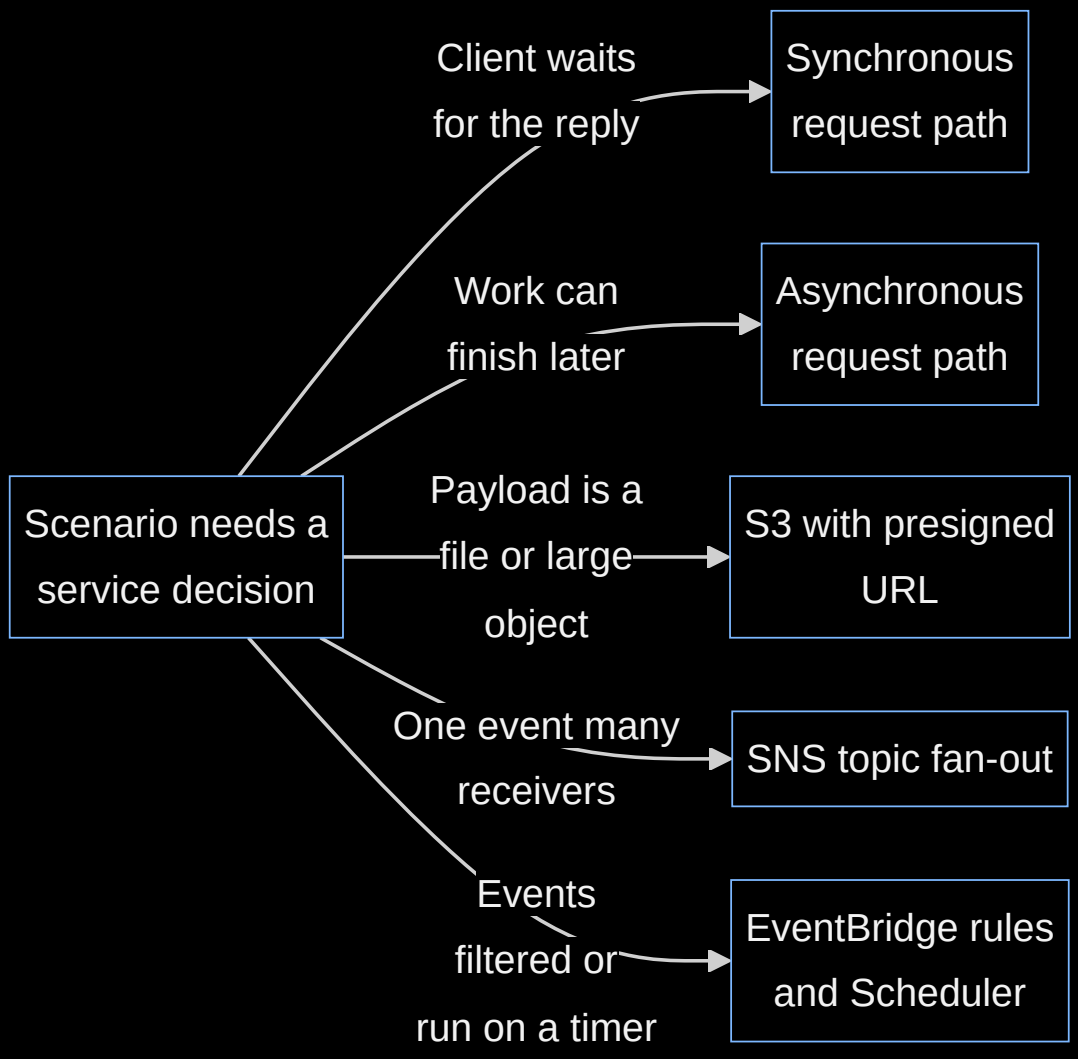


Figure: the first split of the deciding constraint tree, from the scenario to a path.

Each branch has signal words. Read the first branch that matches a strong signal, then check the others for a second constraint.

- Exam tip: Waiting words such as "waits", "as it is generated", or "live" point to the synchronous path.
- Exam tip: Words such as "overnight", "batch", "job ID", or "no user waits" point to the asynchronous path.
- Exam tip: Words such as "upload", "PDF", "video", or "larger than the payload limit" point to S3 with a pointer or presigned URL.
- Exam tip: Words such as "every subscriber", "several teams", or "each consumer receives" point to fan-out.

- Exam tip: Words such as "every weekday", "cron", or "when a field equals" point to EventBridge.

Synchronous branch in detail

A synchronous path keeps the caller on an open connection until the reply is ready. The shape of the reply decides which API type fits. A plain reply uses a request-response API. A reply that arrives in pieces needs a streaming path. A session with messages in both directions needs a WebSocket.

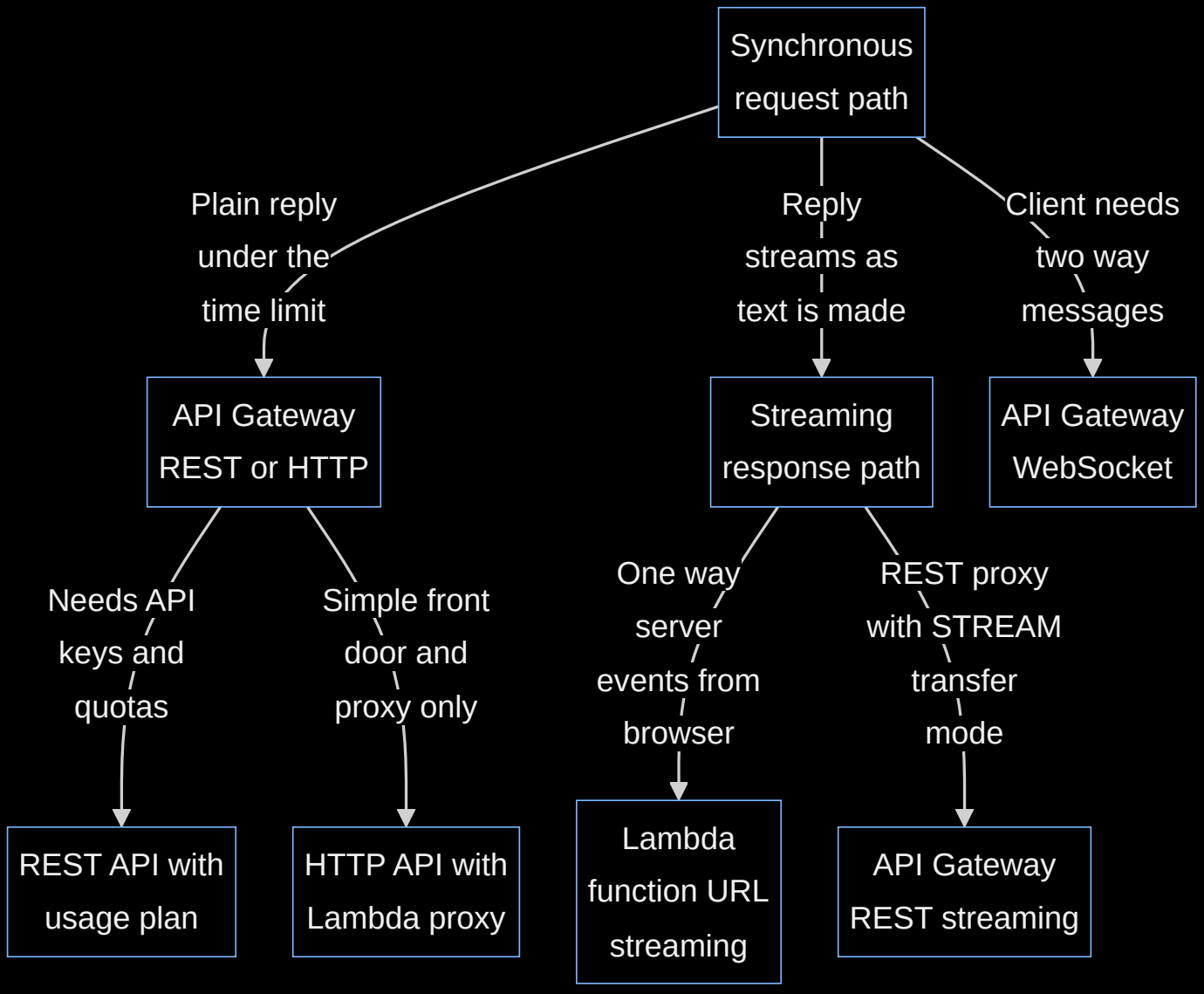


Figure: the synchronous branch, split by reply shape and client needs.

Rule: If the reply must stream through API Gateway, pick a REST API with a proxy integration and STREAM transfer mode. The fetched API Gateway page limits response streaming to REST APIs. An HTTP API is the wrong answer for that requirement, even when the rest of the scenario fits it.

Which limit breaks first

A request crosses several hops. Each hop has its own limit. The first limit that the reply exceeds is the one that breaks, and the others never matter for that request. For a slow model call, the front door usually breaks first.

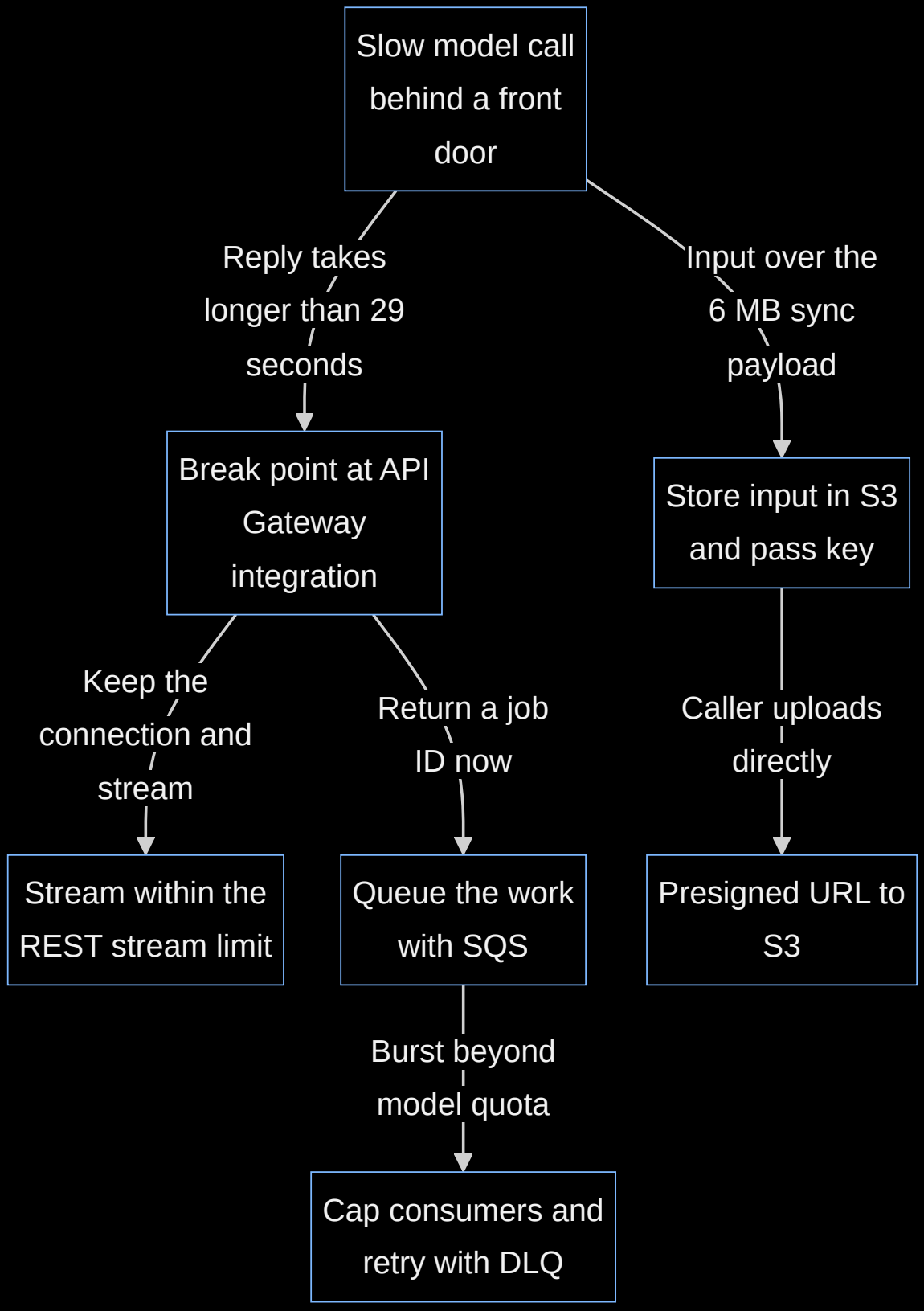


Figure: where a slow model call breaks first, and the fix for each break point.

The fetched API Gateway page says that streaming lets a response run past the 29-second integration timeout without a quota increase. The Lambda timeout is 15 minutes, so a longer Lambda timeout does not help a caller behind a buffered REST

integration. Lambda may keep running after the client connection closes, and the client still sees a failure.

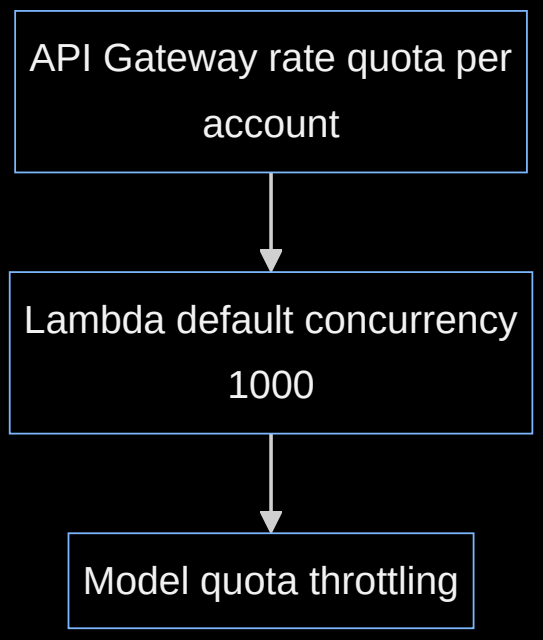


Figure: the order of three rate limits for a burst of model calls.

Trap: Reading the 15-minute Lambda timeout as the answer to a slow API call. The request still ends at the API Gateway integration limit unless the response streams or the work runs asynchronously.

Services in scope

API Gateway

Fact: API Gateway fronts REST, HTTP, and WebSocket APIs. Integrations can point to Lambda, an HTTP backend, or a direct AWS service call. Course slide 358 and 359 list these features.

Rule: Choose API Gateway when clients need a managed front door with authentication, throttling, and request validation before the model call.

Trap: Using API Gateway to hold a long reply open with a buffered integration. The integration limit of about 29 seconds applies unless the response streams.

Limits: The account throttle is 10,000 requests per second per Region, with a burst bucket of up to 5,000 requests. The response payload limit is 10 MB, and streaming can exceed it. Usage plan and API key numbers: Not in source. WebSocket connection duration and idle timeout: Not in source.

Pairs with Cognito authorizers, Lambda proxy integrations, and WAF on the public edge. The course slide also lists request validation and API keys as reasons to use the front door.

Why this matters: API Gateway is the first place that a slow reply meets a time limit. The front door decides whether a streamed reply, a buffered reply, or a queued job is possible. Throttling and API keys at this layer protect the model quota before traffic reaches Lambda.

Source: https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html and https://docs.aws.amazon.com/apigateway/latest/developerguide/response-transfer-mode.html. Course slide extract, pages 358 to 362.

Lambda

Fact: A Lambda invocation can run up to 15 minutes. A function can use 128 MB to 10,240 MB of memory. Memory also sets the CPU share, and 1,769 MB equals one vCPU.

Rule: Use Lambda for the code around a model call when each call finishes inside the timeout and the payload limits.

Trap: Raising the function timeout to fix a slow API call. The client still sees a timeout at the API layer.

Lambda response streaming works through function URLs, the InvokeWithResponseStream API, or the API Gateway proxy integration. The fetched page says native streaming works on Node.js managed runtimes. Python needs a custom runtime or the Lambda Web Adapter. Streamed responses are billed for the full duration, and they are not stopped when the client disconnects.

Limits: Synchronous request and response payloads are 6 MB each. Asynchronous payloads are 1 MB. A streamed response can reach 200 MB. The first 6 MB of a stream is uncapped, then the rate is 2 MBps.

Limits: Default concurrency is 1,000 per Region. Cold start duration: Not in source.

Pairs with API Gateway, SQS event sources, Kinesis batches, and CodeDeploy traffic shifts on Lambda aliases. The course slide lists the canary and linear shift options for Lambda aliases (slide 388).

Why this matters: the default concurrency of 1,000 sets how many model calls can run at once. A burst of requests therefore needs a queue or a reserved cap. The 6 MB synchronous payload is the reason that large model inputs move to S3 first.

Source: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html and https://docs.aws.amazon.com/lambda/latest/dg/configuration-response-streaming.html.

S3

Fact: S3 stores documents as objects under keys. Object creation events can start a Lambda function, and S3 events can appear as an EventBridge rule source (course slides 48 and 407).

Fact: A presigned URL grants time-limited access to one object and one HTTP method. It has the permissions of the signer. A PUT with the same key replaces the existing object.

Rule: Give the client a presigned URL when the client should move bytes straight to or from S3 and the API path should stay small.

Trap: Expecting a presigned URL to outlive its signer. It expires when the role session or temporary credential expires, even if the URL shows a later time.

Limits: The console allows 1 minute to 12 hours. The CLI and SDKs allow up to 7 days. An AssumeRole session defaults to 1 hour in the fetched FAQ. Object Lock retention modes and lifecycle timing: Not in source.

Pairs with Lambda for event handling, EventBridge for routing, and the knowledge base sync step. The course slide shows an S3 event that starts an ingestion job through Lambda.

Why this matters: a presigned URL keeps the object path out of the API and out of Lambda. The client gets rights to one key, and the signer keeps the underlying permissions. That is why the pattern fits uploads of large files that need a short-lived grant.

Source: https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html. Course slide extract, pages 48, 190, and 407.

DynamoDB

Fact: Time to Live reads a Number attribute in Unix epoch seconds. Expired items are deleted by the system, typically within a few days after expiry. Deleted items go to

DynamoDB Streams as service deletions, and they leave the local and global secondary indexes.

Rule: Store chat turns with a session key and a TTL attribute when the history must expire without a cleanup job.

Trap: Treating TTL as an exact expiry timer. An expired item can still come back from a scan or query until the system deletes it, so the read path needs a filter.

Trap: Confusing DAX with TTL. The course slide gives a 5-minute default TTL for the DAX cache, which is a cache setting and not item expiry.

Limits: DAX has up to 10 nodes in a cluster, and it gives microsecond cached reads (slide 182). Item size limit and table capacity: Not in source.

Pairs with Lambda for reads and writes, DAX for read-heavy keys, and OpenSearch for vector search. The course slide shows a zero-ETL path from DynamoDB to OpenSearch (slide 188).

Why this matters: a conversation history table grows without limit unless something removes old turns. TTL is the managed answer when the expiry rule is a time after the last write. A rule that needs removal at an exact moment needs a check in the read path or a different store.

Source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html. Course slide extract, pages 182 and 188.

SQS

Fact: SQS keeps a message until a consumer deletes it. Standard queues give at-least-once delivery and best-effort ordering. Consumers receive up to 10 messages per call (course slides 399 to 401).

Rule: Put a queue between the caller and a model call when bursts exceed the model quota or the caller should not wait.

Trap: Setting the visibility timeout shorter than the model call. The message reappears while the first worker still runs, so a second worker processes the same job.

Limits: Maximum message size is 1 MiB. Larger payloads use the Extended Client Library with an S3 reference, up to 2 GB. Retention defaults to 4 days, with a maximum of 14 days.

Limits: Visibility timeout defaults to 30 seconds, with a maximum of 12 hours. Delay is up to 15 minutes. A batch holds 10 messages.

Dead-letter queues: The maxReceiveCount sets how many receives happen before a move to the DLQ. The DLQ has to live in the account and Region of its source queue. The redrive allow policy can name up to 10 source queues. For a standard queue, set the DLQ retention longer than the source retention, because the original enqueue time is kept.

Rule: Add a dead-letter queue when failed model jobs must stay for inspection or a later replay.

Trap: Setting maxReceiveCount to 1. One transient throttle then sends a good job to the DLQ.

Pairs with Lambda event source mappings, SNS fan-out to one queue per consumer, and Kinesis when the records need replay.

Why this matters: the visibility timeout is the lease that a worker holds on a job. A model call that runs longer than the lease creates duplicate work, so the lease must cover the slowest call plus a margin. The dead-letter queue turns silent retries into a list that people can inspect.

Source: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/quotas-messages.html and https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html. Course slide extract, pages 399 to 404.

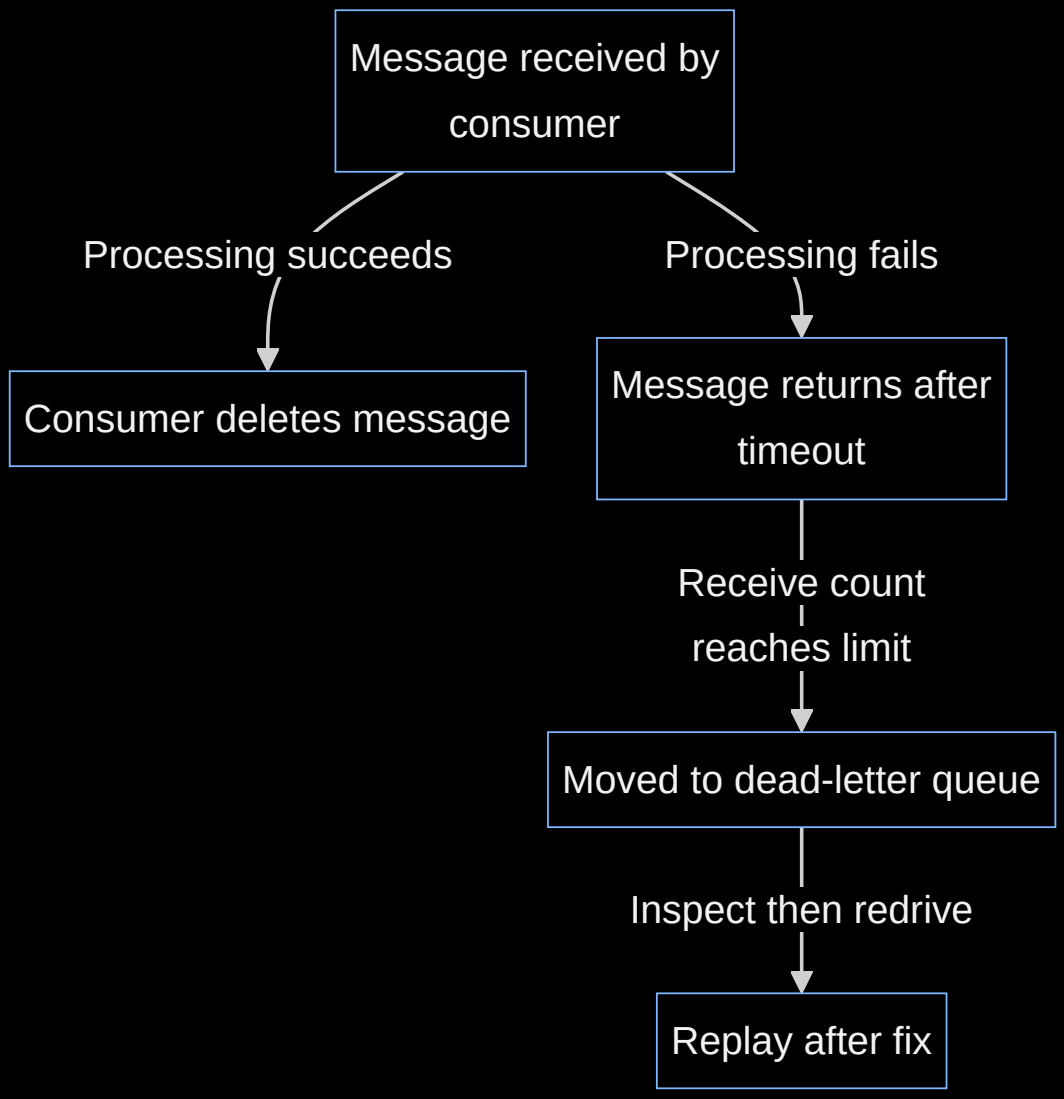


Figure: the path of a failed SQS message to the dead-letter queue and back.

SNS

Fact: SNS takes one publish and sends a copy to every subscriber. Subscriber types include SQS, Lambda, Firehose, email, SMS, and HTTP(S) endpoints (slide 412).

Rule: Use SNS when one event must reach several receivers at once, such as an alert to operations and an index job at the same time.

Trap: Using SNS as the buffer for slow model calls. SNS delivers to subscribers, so put an SQS queue behind each consumer to hold its backlog.

Limits: Up to 12,500,000 subscriptions per topic and 100,000 topics (slide 412). Message retention inside SNS: Not in source.

Pairs with SQS for each consumer, with Lambda for simple handlers, and with EventBridge when the producer needs filters on event fields.

Source: course slide extract, pages 411 to 415. No AWS page fetched for SNS.

EventBridge

Fact: EventBridge has event buses that route events to targets, pipes for one source to one target with enrichment, and Scheduler for cron, rate, and one-time schedules. Scheduler can set retry limits and the maximum retention for failed invocations.

Fact: The Custom Event Bus keeps events for a set period, supports replay and ordered delivery, and accepts non-JSON payloads. New applications should start with the Custom Event Bus. The Classic bus uses rules and targets only.

Rule: Use a schedule for a timed job and an event rule for a reaction to a service event.

Trap: Building a timer from a Lambda loop or a rule with no retry plan. Scheduler is the managed timer, and it holds the retry settings.

Limits: Rule count, targets per rule, and event size: Not in source. The course slides cover a schema registry for inferred event schemas and a resource-based policy for cross-account buses (slides 409 and 410).

Pairs with S3 events, DynamoDB Streams through Pipes, SQS, and Step Functions as targets.

Why this matters: an event bus decouples the producer from its consumers, so a new consumer does not change the producer. Scheduler removes the need for a timer process in your own code. Pipes reduce glue code when the source and the target are fixed.

Source: https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html. Course slide extract, pages 406 to 410.

Kinesis Data Streams

Fact: Kinesis Data Streams collects real-time records. Retention runs up to 365 days, and consumers can replay records within that window. Records are up to 1 MB, and ordering holds for records with the same partition key (slide 543).

Rule: Choose Kinesis for ordered, high-volume records that several consumers read or replay.

Trap: Using Kinesis as a one-time job queue. Records stay until they expire, so each consumer tracks its own position.

Limits: Lambda reads batches of up to 10,000 records, with batches split beyond the 6 MB payload limit. A failing batch is retried until it succeeds, and it can stall the shard (slide 355). Shard throughput numbers: Not in source.

Pairs with Lambda consumers, Firehose for delivery to storage, and MSK when the team already runs Kafka (slides 546 to 548).

Source: course slide extract, pages 355 and 542 to 548. No AWS page fetched for Kinesis limits.

Glue and DataBrew

Fact: A Glue crawler scans S3, infers the schema, and stores table definitions in the Glue Data Catalog. The data stays in S3 (slide 104). Glue Studio builds visual ETL jobs with sources such as S3, Kinesis, Kafka, and JDBC, and it writes to S3 or the Catalog (slide 106).

Rule: Use a crawler to record a schema, and use an ETL job to change the data. Use Comprehend inside Lambda for PII checks before a prompt (slide 118).

Trap: Running a crawler to flatten nested records. A crawler reads the schema only, and it does not write reshaped data.

DataBrew: Features and limits: Not in source. Treat DataBrew as a visual preparation option only when the scenario says so, and check the AWS docs before the exam.

Pairs with Lake Formation for table and column permissions, Athena for queries, and Kinesis for streaming ETL.

Source: course slide extract, pages 104, 106, and 118. No AWS page fetched for Glue or DataBrew.

Textract and Transcribe

Fact: The course slide places Transcribe in the data preparation group, next to Comprehend (slide 5). Textract and Transcribe limits and features: Not in source.

Purpose from general service knowledge, not from a fetched page: Textract reads text and fields from scanned documents, and Transcribe turns speech into text.

Rule: Put Textract or Transcribe first in an ingestion pipeline when the input is not text yet.

Trap: Sending scanned PDFs straight to a model. The model sees no extracted text, so the answer has no source content.

Pairs with S3 events for the start of the pipeline, Glue for cataloging, and Comprehend for PII redaction.

Source: course slide extract, page 5 and page 118. No AWS page fetched for Textract or Transcribe.

AppSync

Fact: AppSync is a serverless GraphQL and pub/sub service. Resolvers can call Lambda, and Apache Velocity Template Language maps GraphQL requests to data sources (slide 391).

Rule: Use AppSync when clients need one GraphQL schema for reads, writes, and live updates.

Trap: Choosing AppSync for a single model call with a fixed REST shape. The GraphQL schema adds work the scenario does not ask for.

Limits: Subscription duration and request quotas: Not in source.

Pairs with Cognito for sign-in, Lambda resolvers for model calls, and WAF on the GraphQL API (slide 508).

Source: course slide extract, pages 391 and 508. No AWS page fetched for AppSync.

Amplify

Fact: Amplify is a set of tools for full stack web and mobile apps. It has frontend libraries, a CLI, a console for hosting and CI/CD, and backend services for auth, storage, API, and AI predictions (slide 405).

Rule: Use Amplify when the question is about building and deploying the app shell and its backend resources.

Trap: Choosing Amplify to design the model workflow or the API contract. Amplify builds the app around the model path, and it does not decide that path.

Limits: Not in source.

Source: course slide extract, page 405. No AWS page fetched for Amplify.

Cognito

Fact: User pools sign in app users and integrate with API Gateway and Application Load Balancer. Identity pools exchange an identity for temporary AWS credentials, and the IAM policies can depend on the user identity (slides 502 to 505).

Rule: Use a user pool for app sign-in. Use IAM roles for internal users who already have AWS access.

Trap: Giving every web user an IAM identity. The course slide names hundreds of users, mobile users, and SAML as signals for Cognito.

Limits: Token lifetimes and user pool quotas: Not in source.

Pairs with API Gateway authorizers, AppSync, DynamoDB row-level access through identity pools (slide 507), and Kendra for document-level access (slide 662).

Source: course slide extract, pages 502 to 507 and 662. No AWS page fetched for Cognito.

CloudFront

CloudFront matters most when the question is about outside users. The edge location absorbs traffic and terminates TLS before the request reaches the API. Put it where the public path starts, not behind the model call.

Fact: CloudFront is the edge content delivery service. The course slides place it in front of a public chatbot, with WAF and API Gateway behind it (slide 663). WAF can attach to CloudFront (slide 508).

Rule: Add CloudFront and WAF when outside users reach the app over the internet.

Trap: Adding CloudFront to an internal-only assistant. The course example skips CloudFront and WAF for internal use (slide 662).

Limits: Not in source.

Source: course slide extract, pages 508 and 662 to 663. No AWS page fetched for CloudFront.

X versus Y decisions

Each pair below has one decision tree, a three-row comparison, the question to ask, and the trap. Ask the question first. Use the comparison only to confirm the answer.

API Gateway REST versus HTTP versus WebSocket

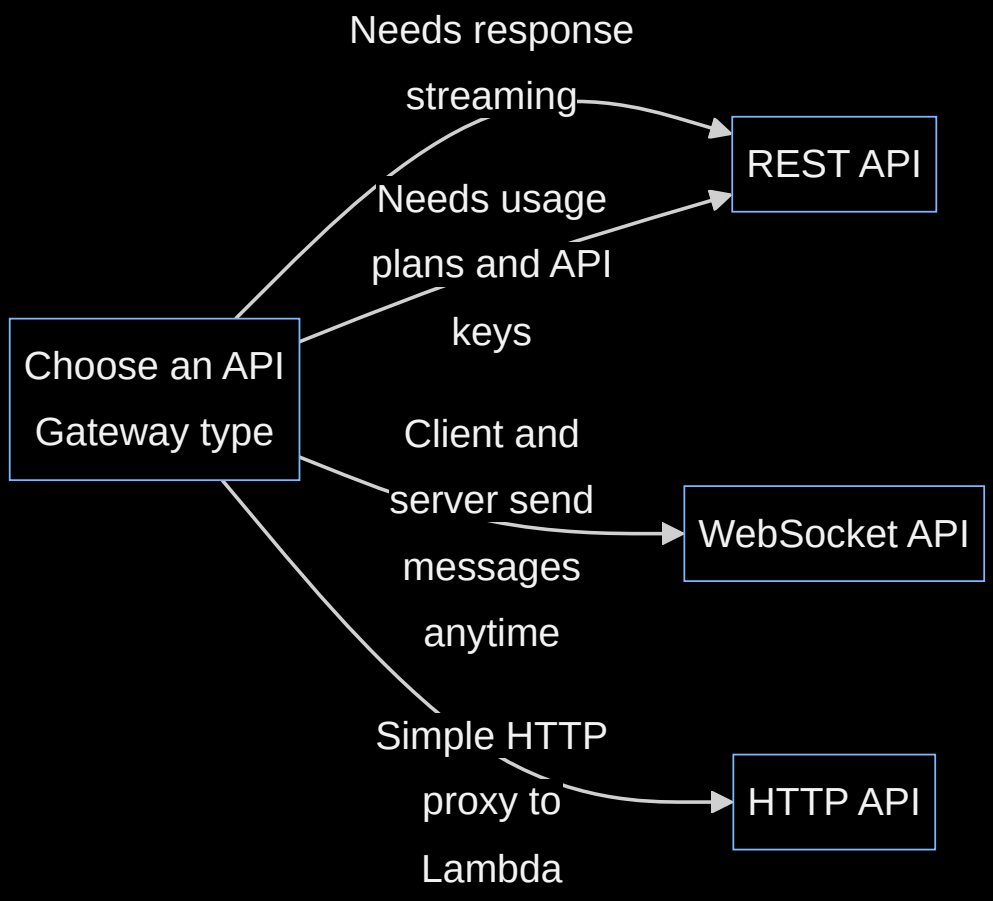


Figure: picking the API type from the feature the scenario needs.

- Purpose: REST for the full feature set, HTTP for a simple proxy, WebSocket for two-way messages.
- Input: REST and HTTP take request-response calls, and WebSocket takes connected clients that send messages.
- Output or effect: The fetched page limits response streaming to REST APIs with a proxy integration.

Exam tip: Find the word that describes the reply shape, then match it to the API type. The reply shape is more decisive than the traffic volume.

The question to ask yourself: does the reply need to stream, or does the client need two-way messages?

Why this pair confuses learners: all three types share the same front door name and accept similar requests. The difference appears in the protocol and in the streaming rule, so the scenario wording carries the answer.

Trap: Picking an HTTP API because it looks simpler, when the scenario needs streamed text through API Gateway. The fetched page says streaming is supported only for REST APIs.

Synchronous versus asynchronous model calls

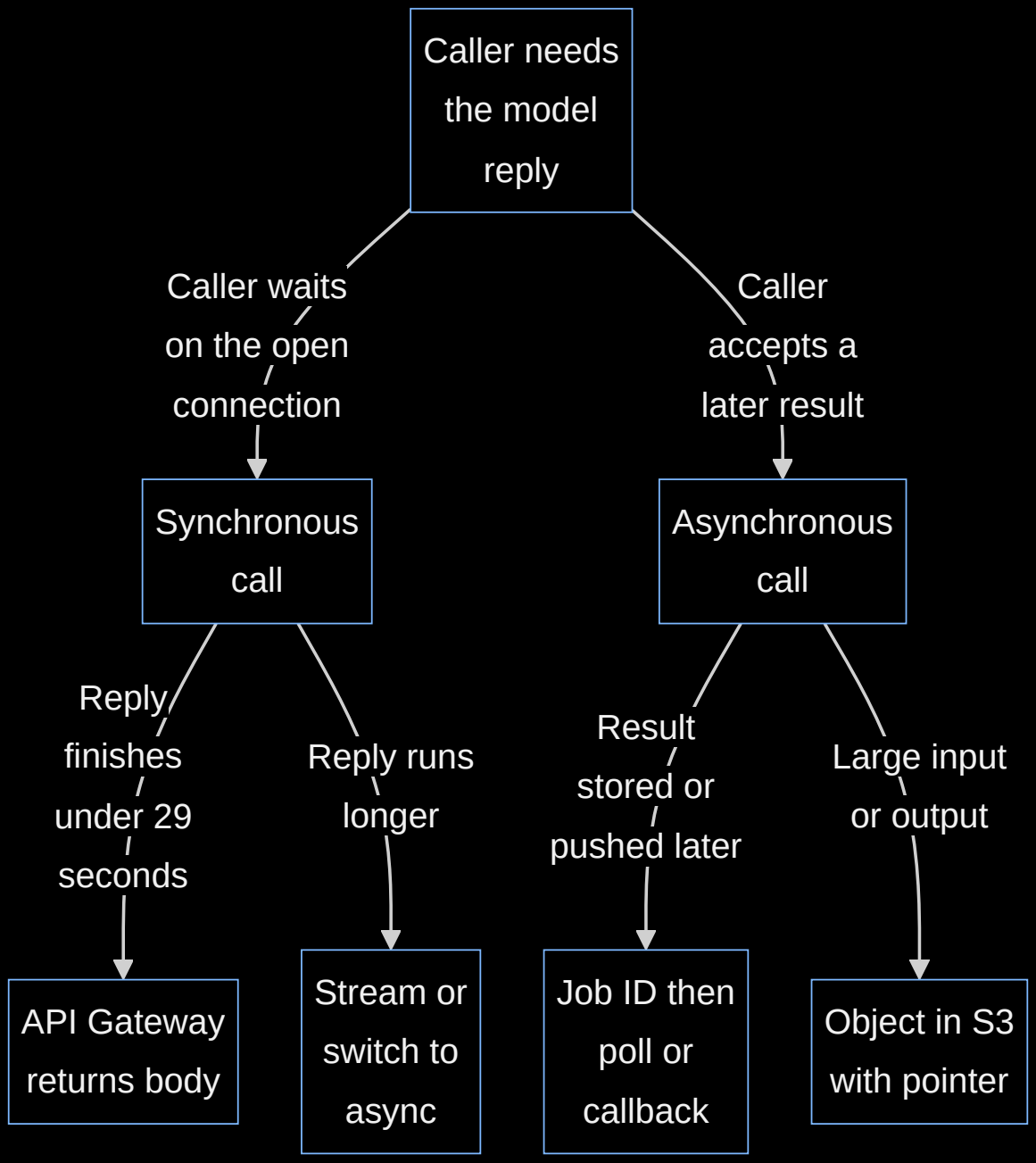


Figure: the choice between a waiting call and a job that returns later.

- Purpose: A synchronous call returns the answer in the same request. An asynchronous call returns a job reference first.
- Input: A synchronous call sends the prompt in the request body. An asynchronous call sends the prompt to a queue or stores it in S3.
- Output or effect: The caller gets a body or a stream now, or a job ID now and the result later.

The question to ask yourself: will the caller still hold an open connection when the model finishes?

Why this pair confuses learners: the synchronous call looks simpler, so learners choose it even when the reply runs past the limit. The asynchronous design adds a job

ID and a result path, which feels like extra work. The time limit decides the pair, so read the wait time before anything else.

Trap: Raising the Lambda timeout to 15 minutes so an API call can wait longer. The API layer still stops the wait at its integration limit, so the client sees a failure.

SQS versus SNS

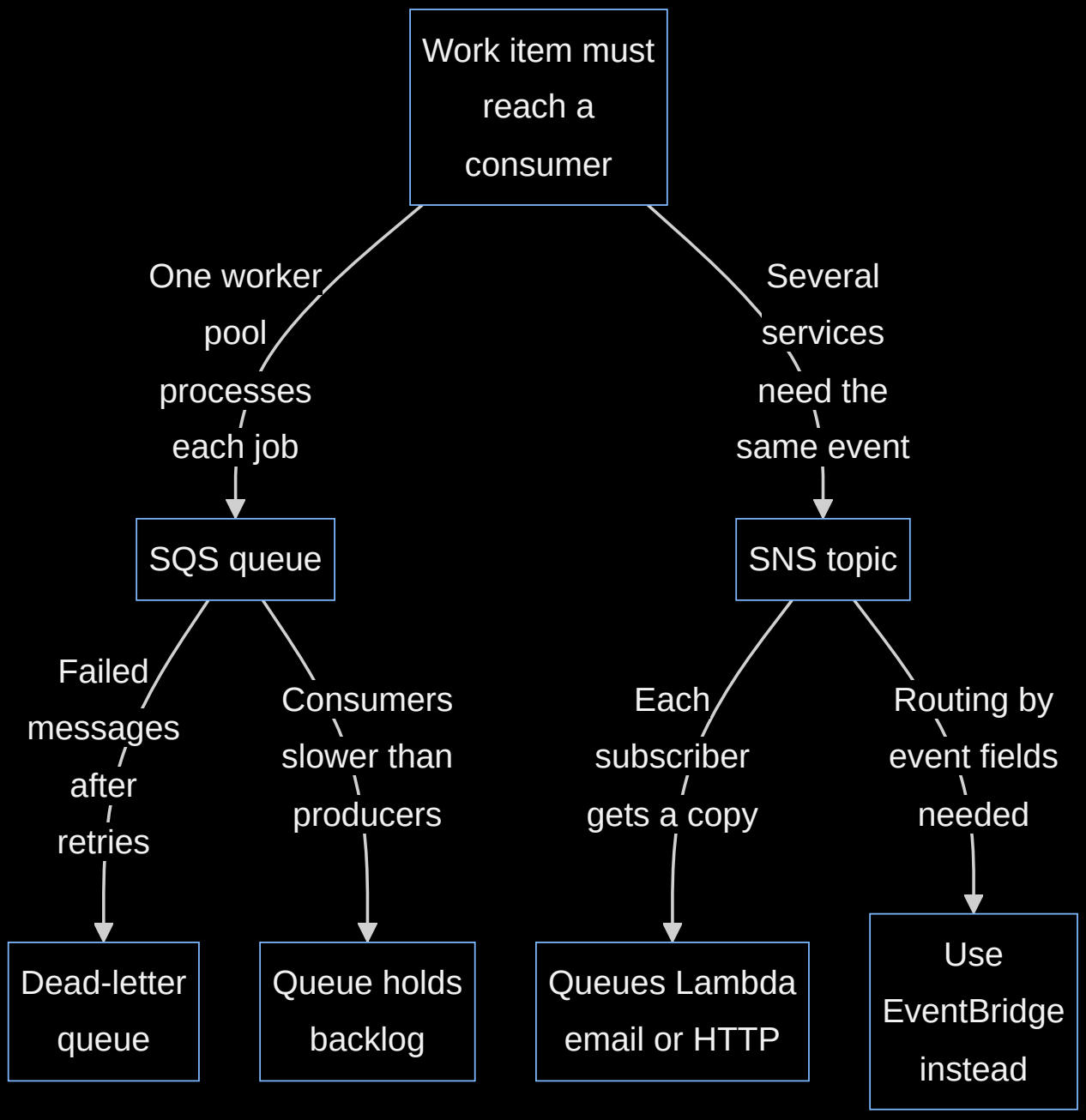


Figure: one job for one pool versus one event for many receivers.

- Purpose: SQS holds work for a consumer group. SNS sends a copy of one message to each subscriber.
- Input: A producer calls SendMessage on SQS. A producer publishes once to an SNS topic.
- Output or effect: SQS removes the message only after a consumer deletes it. SNS delivers a copy to each subscriber.

The question to ask yourself: is this one job for one worker pool, or one event for many receivers?

Why this pair confuses learners: both move messages between parts of the app. SNS sends one copy to each of many receivers, while SQS keeps one copy for one group of workers. Count the consumers before you choose.

Trap: Using SNS as the backlog for a slow model consumer. The course slide describes SQS as the holder of messages until deletion, so the backlog belongs in a queue after the topic.

SNS versus EventBridge

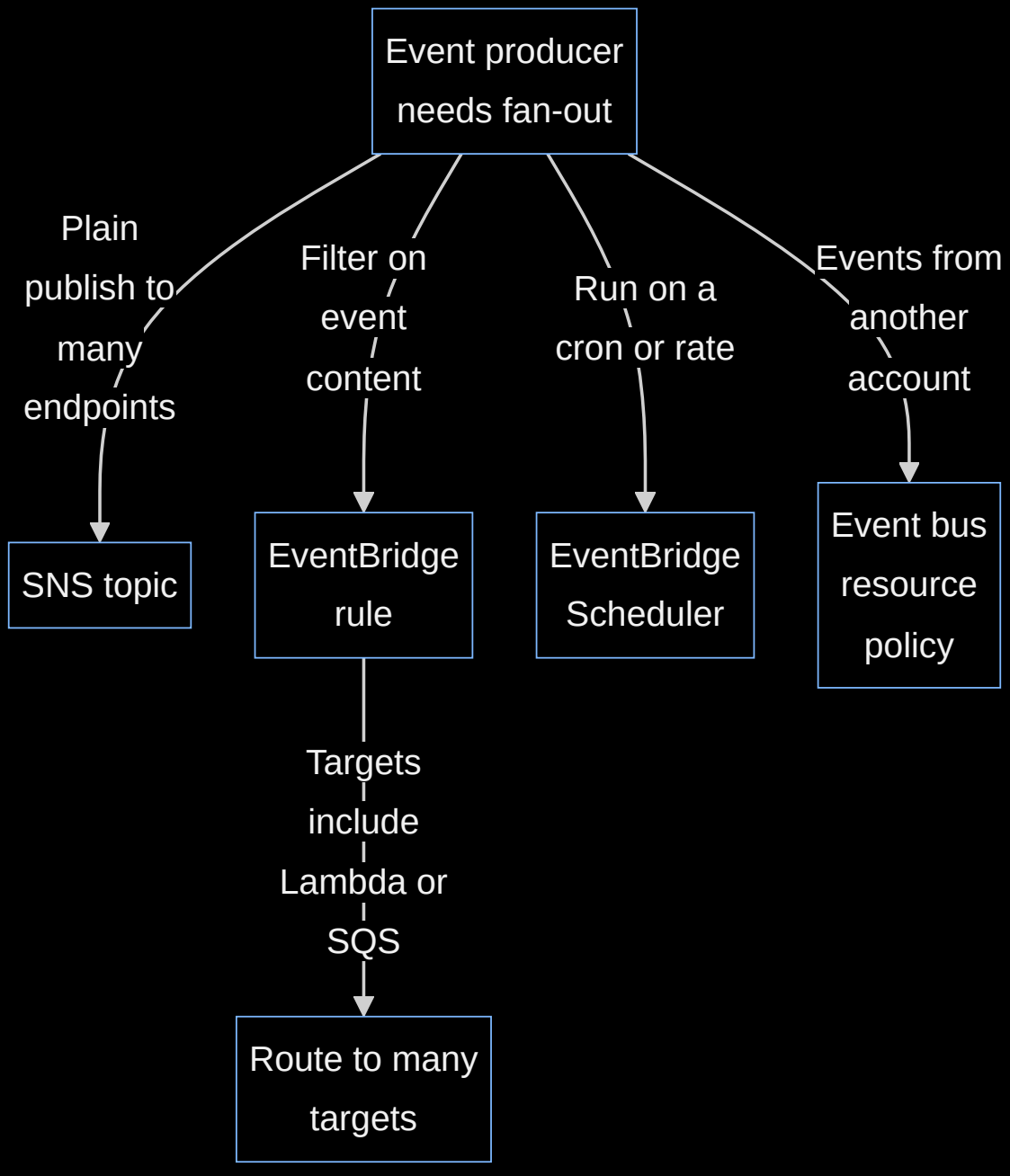


Figure: plain broadcast versus content routing and scheduling.

- Purpose: SNS broadcasts to subscribers. EventBridge routes events by pattern and runs schedules.

- Input: SNS takes a publish call. EventBridge takes events from AWS services, applications, and schedules.
- Output or effect: SNS sends copies to endpoints. EventBridge sends matching events to each target that the rule names.

The question to ask yourself: must the router match fields in the event, or run on a timer?

Trap: Writing a Lambda cron to publish to SNS on a schedule. Scheduler is the managed timer, and it needs no extra publishing code.

EventBridge rule versus Pipes

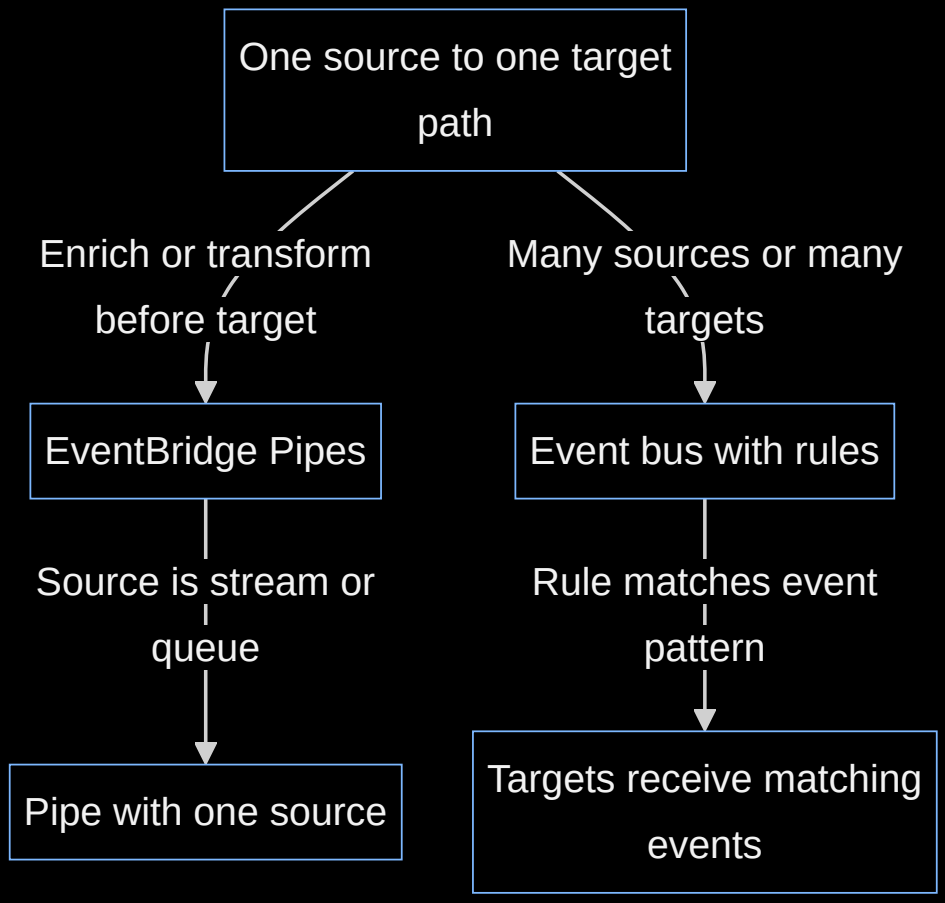


Figure: a single point-to-point path versus a routed bus.

- Purpose: A rule routes to several targets. A pipe connects one source to one target and can enrich the event.
- Input: A rule reads events on a bus. A pipe reads from one source such as a stream or a queue.
- Output or effect: Each matching target gets the event from a rule. The single pipe target gets the transformed event.

The question to ask yourself: is there one source and one target with a transform step between them?

Why this pair confuses learners: both connect an event source to a target. A pipe is a point-to-point integration, so one source and one target are the signal. A bus fits many sources or many targets, and the rules on the bus do the routing.

Trap: Using a pipe as the router for many consumers. A pipe has one target, so a bus with rules is the fan-out choice.

Kinesis versus SQS

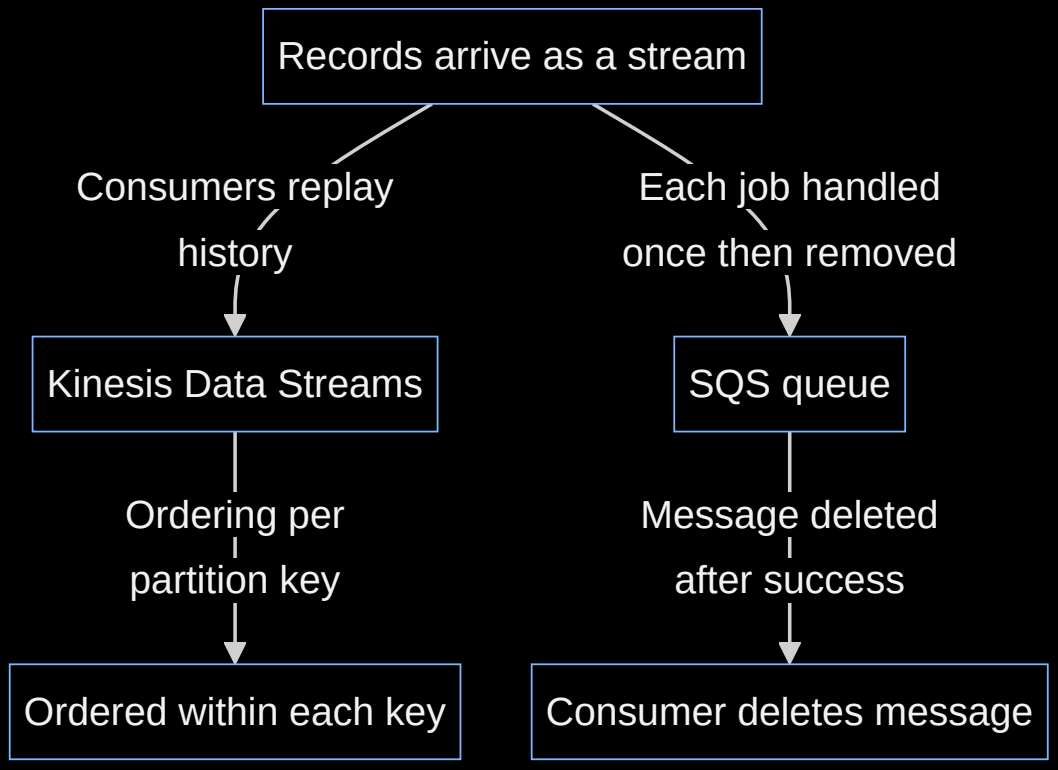


Figure: replayable ordered records versus one-time jobs.

- Purpose: Kinesis keeps an ordered record history for many readers. SQS holds jobs for one consumer group.
- Input: Producers write records with a partition key to Kinesis. Producers send messages to SQS.
- Output or effect: Each Kinesis reader keeps its position and can replay. An SQS message leaves the queue after a delete.

The question to ask yourself: will more than one consumer read the same records, or will someone replay them later?

Why this pair confuses learners: both hold data while a consumer is busy. Kinesis keeps an ordered history for replay, and SQS removes work after a successful delete.

The word replay is usually the strongest signal in the scenario.

Trap: Choosing SQS when the scenario asks for records from last week to be read again. The message is gone after the consumer deletes it.

Glue crawler versus Glue ETL job

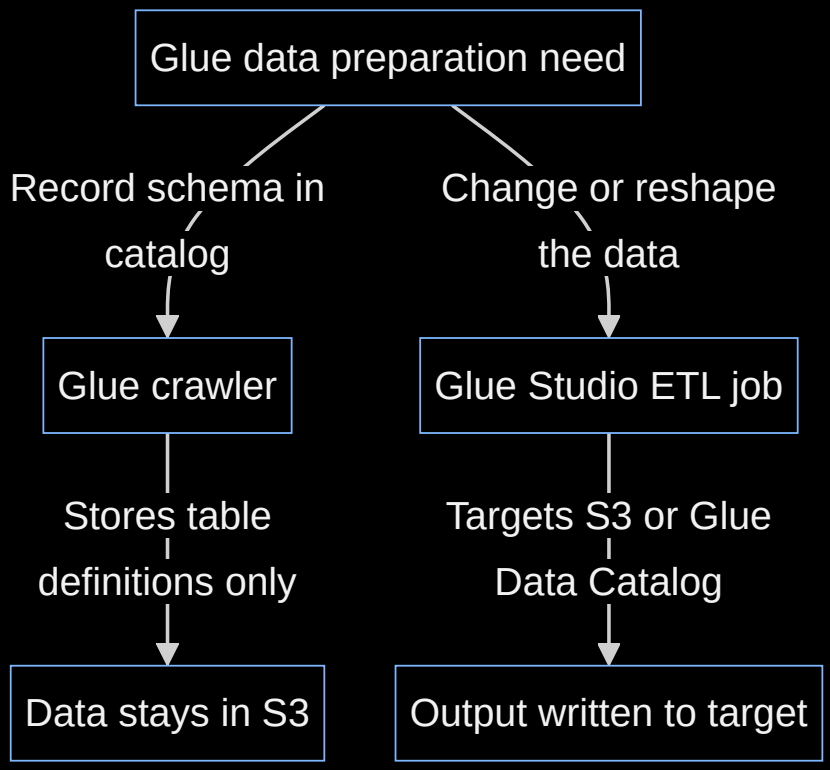


Figure: schema capture versus data transformation in Glue.

- Purpose: A crawler captures the schema. An ETL job transforms the data.
- Input: A crawler reads S3 objects. An ETL job reads its configured sources.
- Output or effect: A crawler writes table definitions to the Catalog. An ETL job writes a new dataset to a target.

The question to ask yourself: is the goal to know the schema, or to change the data?

Why this pair confuses learners: both sit in the same Glue family and both read the same S3 data. The crawler creates metadata, and the ETL job creates new data. Ask which output the scenario needs before you read the options.

Trap: Using a crawler to flatten nested JSON for a knowledge base. Only the ETL job writes changed data.

Textract versus Transcribe

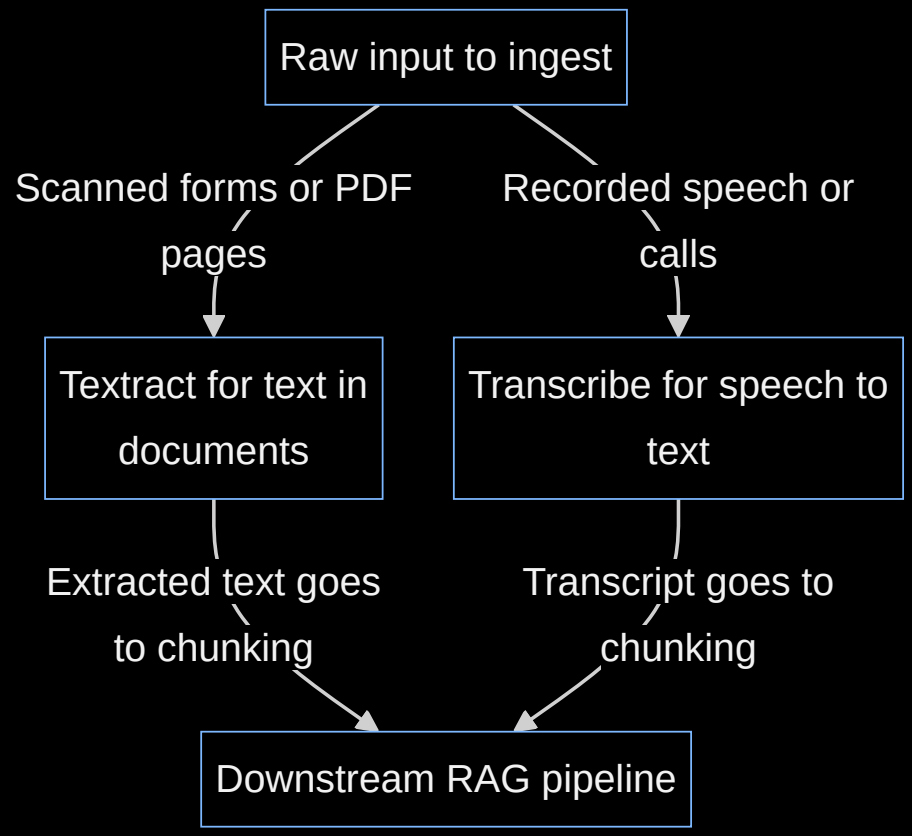


Figure: the first extraction step, chosen by the type of raw input.

- Purpose: Textract reads text and fields from documents. Transcribe converts speech to text. Purpose is from general service knowledge, not a fetched page.
- Input: Textract takes scanned pages or images. Transcribe takes audio.
- Output or effect: Textract returns text and fields. Transcribe returns a transcript.

The question to ask yourself: is the raw input a document image or a recording?

Why this pair confuses learners: both turn raw content into text, so the output looks the same. The input type decides the service. A scanned bill is a page image, and a call recording is audio.

Trap: Choosing Transcribe for scanned bills. Transcribe takes audio, so the page images never reach it as speech.

AppSync versus API Gateway for a front end

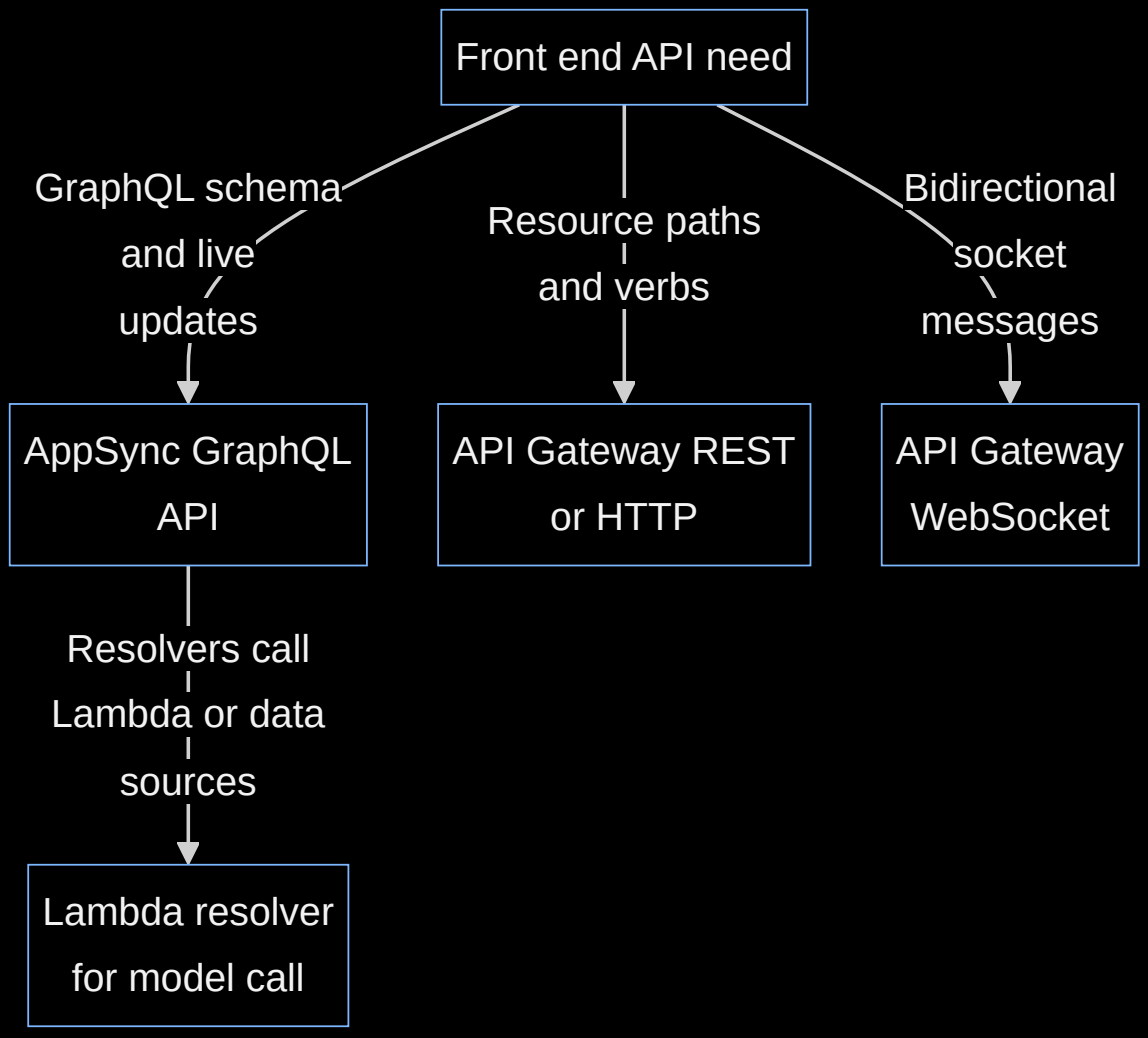


Figure: choosing the front-end API type from the client data shape.

- Purpose: AppSync gives a GraphQL schema with subscriptions. API Gateway gives a resource API with a managed front door.
- Input: AppSync takes GraphQL queries, mutations, and subscriptions. API Gateway takes HTTP methods or socket messages.
- Output or effect: AppSync returns graph-shaped data and pushes updates. API Gateway returns HTTP responses or socket frames.

The question to ask yourself: does the client need a typed graph with live subscriptions?

Why this pair confuses learners: both serve a web or mobile front end. GraphQL with subscriptions is the feature that separates them. A scenario that asks for live updates to a typed graph points to AppSync, and a scenario that names resource paths points to API Gateway.

Trap: Choosing AppSync because the team uses a React front end. The front-end framework does not decide the API type.

Cognito user pools versus identity pools

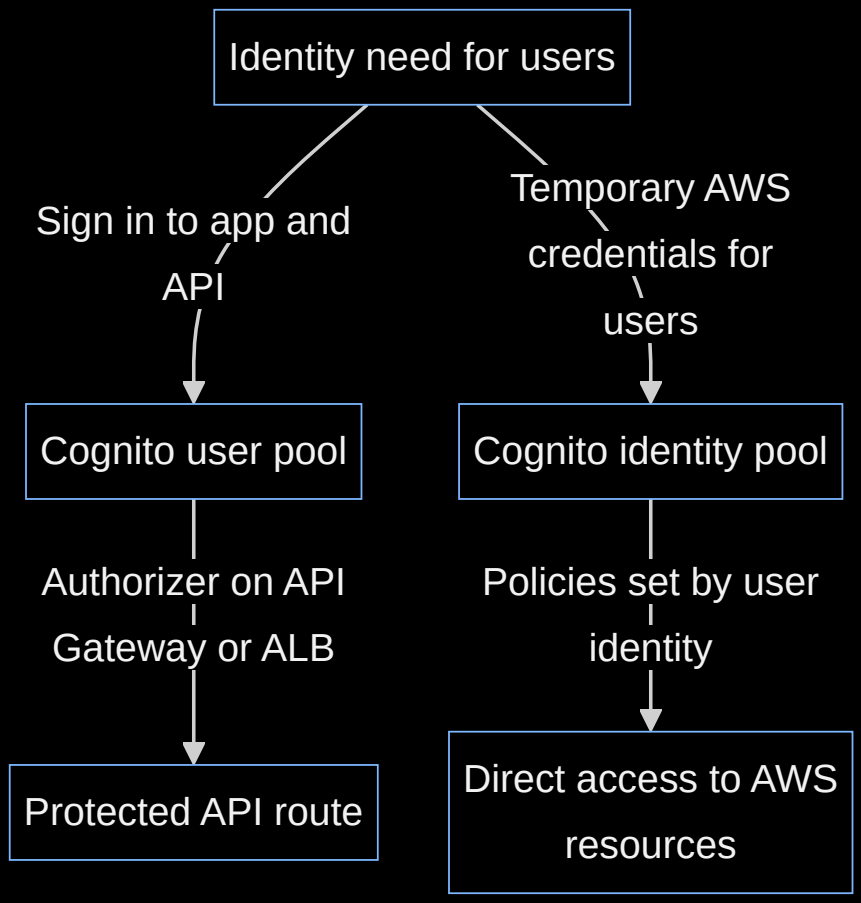


Figure: app sign-in versus direct AWS access for an authenticated user.

- Purpose: A user pool signs app users in. An identity pool grants AWS credentials.
- Input: A user pool takes app sign-in with a user directory or a federated login. An identity pool takes an identity from a user pool or a third party.
- Output or effect: A user pool gives an API authorizer a token. An identity pool gives temporary AWS credentials under IAM policies.

The question to ask yourself: does the user need an API token, or direct AWS access?

Why this pair confuses learners: both belong to the same identity service family. The first gives an app a sign-in result for its own API. The second gives a user temporary AWS access, and the IAM policy decides what that user can touch.

Trap: Giving a browser identity pool credentials to call a model directly when the app must apply business checks first.

Presigned URL versus Lambda upload

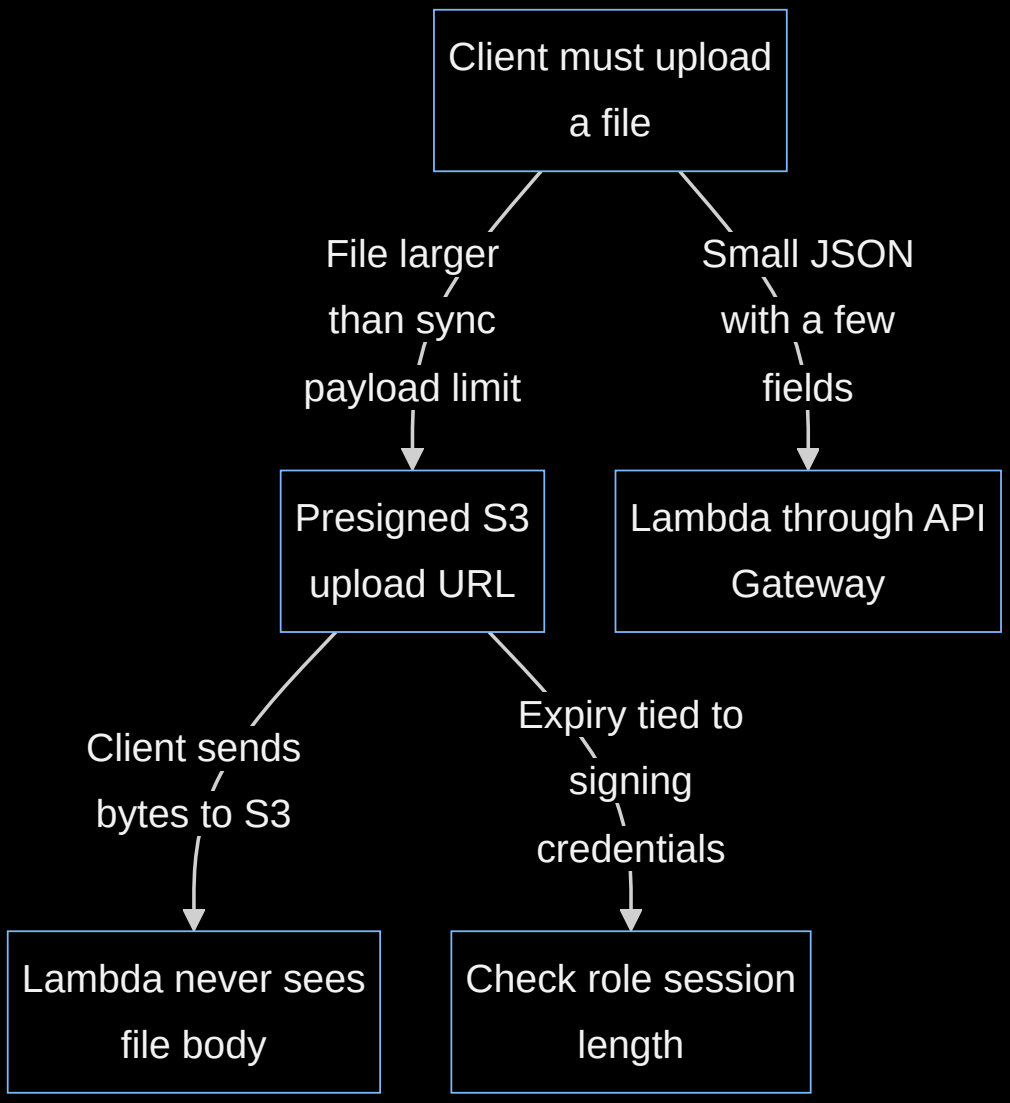


Figure: the upload path for a large file versus a small request body.

- Purpose: A presigned URL moves a large object straight to S3. A Lambda upload passes a small body through the function.
- Input: A presigned URL goes to the client. A Lambda upload sends the body through API Gateway.
- Output or effect: The presigned path keeps the file out of the Lambda payload. The Lambda path is limited to the synchronous payload size.

The question to ask yourself: does the file body need to pass through the API at all?

Why this pair confuses learners: both let a client send a file to storage. The difference is the path the bytes take. Payload size and signer lifetime decide the path, so check both numbers in the scenario.

Trap: Handing out a presigned URL with a long expiry, expecting it to outlive a role session that ends in one hour.

Read a long question

Each worked scenario below is my own. Constraint words are in bold. Each wrong option is removed by one reason. The answer is the option that keeps every constraint.

Scenario one: streamed text over one HTTP path

A health information portal shows a chat answer in the browser as it is written. The front end already calls one request-response path. Some answers take about 40 seconds, and the team wants to **keep the request-response shape**.

Constraint words: **as it is written**, **request-response shape**, **40 seconds**.

- Buffered REST API with a long Lambda timeout: removed, because the 29-second integration limit still ends the request.
- HTTP API with proxy streaming: removed, because the fetched page limits response streaming to REST APIs.
- WebSocket API: removed, because it changes the protocol that the scenario says to keep.
- REST API with a proxy integration set to STREAM, and a streaming Lambda: kept, because it meets streaming and the request shape.

Answer: the REST API with STREAM transfer mode and a streaming Lambda function.

Why it wins: it is the only option that keeps the reply open past the integration limit while it keeps the HTTP request shape. Each other option fails one named constraint.

Scenario two: overnight work with failures kept

A claims team sends about 80,000 records each night for a model summary. No user waits, and each failed record must remain for a later replay. Each record is about 300 KB, and the model has a per-minute quota.

Constraint words: **no user waits**, **failed record must remain**, **per-minute quota**.

- One synchronous API call per record: removed, because the 29-second limit and the per-minute quota break the run.
- SNS topic with one Lambda subscriber: removed, because SNS sends to subscribers and does not hold a backlog for a slow consumer.
- One scheduled Lambda that loops over every record: removed, because one invocation has a 15-minute timeout that may not cover the run.

- SQS standard queue with a dead-letter queue and a Lambda consumer with capped concurrency: kept, because it holds the backlog, keeps failures, and limits the rate. The 300 KB record is under the 1 MiB message limit.

Answer: the SQS queue with a dead-letter queue and a capped Lambda consumer.

Why it wins: the queue holds the backlog, the dead-letter queue keeps failures for replay, and the consumer cap respects the model quota. No user waits, so the asynchronous design costs nothing that the scenario values.

Scenario three: a timed job and an upload trigger

A publisher must run a summary job at 07:00 each weekday. Editors also upload files to S3, and each upload must start a review job. The team wants no servers for the timer.

Constraint words: **07:00 each weekday**, **each upload**, **no servers for the timer**.

- SNS topic as the timer: removed, because SNS does not run on a schedule.
- Kinesis stream for uploads: removed, because S3 object events are not a record stream that producers write.
- DynamoDB TTL as the trigger: removed, because TTL deletes items at an unfixed later time and does not run on a clock.
- EventBridge Scheduler for the timer, plus an S3 event into an EventBridge rule for uploads: kept, because it meets both triggers with managed services.

Answer: EventBridge Scheduler for the weekday job, and an EventBridge rule on S3 upload events.

Why it wins: each trigger has its own managed source, and no code runs as a timer. The scenario named two triggers, and this is the only option that covers both.

Distractor patterns seen in the tests

These patterns repeat across the practice scenarios. Each one is a real service used at the wrong layer. Learn the pattern, and you can reject the option even when the service name is new to you.

Each pattern works because the scenario hides a second constraint in one line. Read the verb in each option first. Storage, transport, schedule, identity, and cache words

each name a different layer, and the option that uses the wrong layer usually sounds the most familiar.

- **Right service, wrong layer.** A Lambda timeout is offered for an API limit. Trap reason: the limit lives at the front door, so a longer function timeout does not move it.
- **Streaming versus protocol change.** WebSocket is offered when the scenario only needs one-way text. Trap reason: the protocol change adds connection state the scenario did not ask for.
- **Broadcast versus backlog.** SNS is offered as a buffer for slow consumers. Trap reason: SNS delivers to subscribers, and the queue behind each subscriber holds the backlog.
- **Delete versus replay.** SQS is offered for replaying old records. Trap reason: a consumer deletes the message after success, so there is nothing left to replay.
- **Cache versus expiry.** DAX is offered for item expiry. Trap reason: DAX is a read cache with its own TTL, and item expiry belongs to the table TTL setting.
- **Identity layer mix-up.** An identity pool is offered for API sign-in. Trap reason: identity pools grant AWS credentials, while user pools give API authorizers a sign-in result.
- **Pipeline order.** A model call comes before text extraction or PII redaction. Trap reason: the model cannot use text that has not been extracted or cleaned yet.
- **Removed control.** An option is cheaper because it drops a dead-letter queue or a retry rule. Trap reason: the scenario names the control, so the option fails that constraint.

Drill

Read each prompt, then cover the answer line. Say the reason before you read the answer.

Prompt: A browser needs text as it is written, over one HTTP path, with no WebSocket.

Rule: Use a REST API with STREAM transfer mode and a streaming Lambda, not an HTTP API.

Prompt: A model reply takes 40 seconds behind a front door. The team plans to raise the Lambda timeout only.

Trap: The 29-second integration limit still breaks first. Stream the reply or switch to an asynchronous job.

Prompt: Several teams need the same order event, each with its own backlog.

Rule: Publish to an SNS topic, and put one SQS queue behind each consumer.

Prompt: One worker pool must process each job once, and failed jobs must be kept.

Rule: Use an SQS queue with a dead-letter queue and a suitable maxReceiveCount.

Prompt: A new consumer next week must read records from last week.

Rule: Use Kinesis Data Streams inside its retention window, not SQS.

Prompt: A browser uploads a 50 MB file for a later review step.

Rule: Give the browser a presigned upload URL, so the bytes skip the Lambda payload.

Prompt: Expired chat turns must stop appearing in a table after 30 days.

Trap: TTL deletion can lag by days. The read path must filter expired items.

Prompt: Web users must sign in before they call an API.

Rule: Use a Cognito user pool with an authorizer on the API.

Prompt: A summary job runs each weekday at 07:00.

Rule: Use EventBridge Scheduler for the timer.

Prompt: A scanned bill must become text before it is chunked for a knowledge base.

Rule: Put Textract first in the ingestion pipeline, then the chunking step.

Prompt: Clients need GraphQL queries and live updates for a chat panel.

Rule: Use AppSync with subscriptions, and Lambda resolvers for the model call.

Prompt: A public chatbot faces the internet and must block abusive traffic.

Rule: Put CloudFront and WAF in front of the API, and keep the API Gateway authorizer.

Sources

Source: AWS documentation fetched for this module.

- https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html
- https://docs.aws.amazon.com/apigateway/latest/developerguide/response-transfer-mode.html
- https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html
- https://docs.aws.amazon.com/lambda/latest/dg/configuration-response-streaming.html
- https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html
- https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html
- https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/quota-messages.html
- https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html
- https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

Source: course slide extract for this module, pages cited in each section. Slide facts are used only where an AWS page did not cover them.